result=data &(1<<bit)

0 : bit in the data is clear

non-zero/2^bit: bit In the data is set

Status of a bit

result= num &(1<<bitposition)

1. num= 5;bitposition=1

```
  0  0  0  0  0  1  0  1   =num
&
  0  0  0  0  0  0  1  0   (1<<bitpos)///(1<<1
-------------------------------------------------
  0  0  0  0  0  0  0  0   =result=0///confirm bit is clear
```

2. num =5, bit position is 2

```
5=  0  0  0  0  0  1  0  1
1<<2 = 0  0  0  0  0  1  0  0
---------------------------------------
  0  0  0  0  0  1  0  0  = result = non-zero, so bit is set in the given data
```

Bitmasking is a technique that involves bit manipulation.
It is basically like putting a mask over certain bits and hiding the other un-useful bits,

==========

SET a bit

result= data | (1<<bit)

we set a particular bit to 1 without touching any of the other bit

so we use bitwise OR and left shit operator(<<)

Q: GIVEN data= 12 , set bit position 1

```
12 =  0  0  0  0  1  1  0  0
      |
1<<1=0  0  0  0  0  0  1  0
-----------------------------------------
      0  0  0  0  1  1  1  0
                   bit is set
```

num=3 ,bit pos =1

```
  0  0  0  0  0  0  1  1
  |
  0  0  0  0  0  0  0  1
-----------------------------------------
  0  0  0  0  0  0  1  1
```

Clearing a bit

data & ~(1<<bitpos)

data =6 ,bit position =2

0 0 0 0 0 1 1 0 =6

1 1 1 1 1 0 1 1 =~(1<<2)

0 0 0 0 0 0 1 0 //result=2

1<<2= 0 0 0 0 0 1 0 0

&

~(1<<3= 1 1 1 1 1 0 1 1

Toggle a bit

data ^ (1<< bitpos)

num =15 ,bit position =3

num = 0 0 0 0 1 1 1 1

^

1<<3= 0 0 0 0 1 0 0 0

---

0 0 0 0 0 1 1 1 ///result=7

---

check whether the given number is even or odd using bitwise operator

check 0 th bit - if 0 : even

1 : odd

0 =0 0 0 0 0 0 0 0

1 = 0 0 0 0 0 0 0 1

2= 0 0 0 0 0 0 1 0 even

3 =0 0 0 0 0 0 1 1 odd

4=0 0 0 0 0 1 0 0 even

5 =0 0 0 0 0 1 0 1 odd

((data & (1<<0)) ==0)?printf("even number ") : printf(" odd number ");

uppercase to lowes case /vice versa

A a a A
65 97 97 65

```
int main()
  char ch;
  scanf("% c ",&ch);///ch= getchar();

(ch>='a ' && ch<='z')   ? printf("%c ", ch - 32)  :  printf("  %c ", ch +32);
```

using bitwise operator

data =data ^ (1<<5)

```
char var='A';
var =var ^ (1<< 5)
```

A =65

7 6 5 4 3 2 1 0
0 1 0 0 0 0 0 1 =65

^ 1<<5

0 0 1 0 0 0 0 0

---

0 1 1 0 0 0 0 0

64 + 32 =97

1:wap to set 0 th bit and 2nd bit in a given number

input: num= 51 , o/p =55

2.WAP to toggle 1st and 4th bit

input: n=42 o/p =56